

Gyal Programming Language

Manual de Referência e Documentação Oficial - Versão 0.0.0.1
Criado por: Omochain

O **Gyal** é uma linguagem de programação de baixo nível projetada especificamente para a criação de Sistemas Operacionais. Ela une a simplicidade e legibilidade da sintaxe do Python com o poder e controle absoluto do Assembly.

1. Variáveis e Tipos

```
var <type> nome = valor
```

Tipo	Descrição
boolean	Verdadeira, Falsa ou Nula.
char	Cadeia de caracteres (String).
hex	Valores hexadecimais puros, essenciais para bootloaders (ex: 0xAA55).
num	Números inteiros, decimais ou constantes matemáticas (ex: pi).

2. Funções e Classes

Funções no Gyal são equivalentes a rótulos (labels) no Assembly, permitindo a organização do código e saltos de execução.

```
func exemplo:  
    var char sysname = "Haicai"
```

3. Comandos de Sistema (Set)

- set mode text - Define o modo de texto padrão (80x25).
- set mode video char <Res> - Modo de vídeo para terminais.
- set mode video graph <Res> - Modo de vídeo para interfaces gráficas.
- set init 0 - Desliga o sistema (Halt).
- set init 1 - Estado de boot (Modo Real).

4. Comandos de Interação Estilo Python

O Gyal herda comandos clássicos do Python para criar terminais interativos, lendo o teclado e exibindo dados na tela.

Print

Exibe informações na tela. O comportamento muda dependendo do modo de vídeo ativo: se estiver em mode text ou mode video char, ele renderiza os caracteres nativamente usando a BIOS. Em mode video graph, ele envia a string para o renderizador da interface gráfica.

```
print "Bem-vindo ao GyalOS!"
```

Input

Lê a digitação do usuário no teclado até que a tecla ENTER seja pressionada, e salva o resultado em uma variável na memória.

```
print "Digite seu nome:"  
input nome_usuario
```

Lógica Condicional (If, Elif, Else)

Essencial para a criação de terminais e interpretadores de comandos. O Gyal usa a **indentação** (espaços no início da linha) para saber onde o bloco de código começa e termina, exatamente como no Python.

```
print "GyalOS Terminal >"
input comando

if comando == "help":
    print "Comandos disponiveis: help, clear, exit"
elif comando == "exit":
    set init 0
else:
    print "Comando nao reconhecido."
```

5. Gerenciamento de Arquivos (Allocate & Nav)

```
allocate create file "tananam.txt"
allocate delete file "tananam.txt"
allocate list "/test"
nav go "/sys"
```

6. Hardware e Processos

```
device scan all
device connect keyboard
async kill <processo>
async list
```

7. Integração e Python

O Gyal permite a inclusão de outros arquivos `.gyal` e scripts `.py` para a interface gráfica, utilizando o interpretador embutido no core da ISO.

```
include "kernel.gyal"
include "desktop.py"
```

Bibliotecas Python suportadas na interface: tkinter, pillow, time, datetime, os, subprocess, numpy, matplotlib, keyboard, pygame.

8. Estrutura de Projetos

- **iso/** - Onde a imagem `.iso` compilada é gerada.
- **sys/** - Arquivos de sistema que serão injetados na ISO.
- **gyal/** - Código fonte (`.gyal`) do bootloader e kernel.
- **src/** - Imagens e áudios codificados em hexadecimal.
- **Config.conf** - Arquivo de configuração do projeto (formato TOML/Rust).